Laporan Tugas Kecil 2

IF2211 Strategi Algoritma
Voxelization Objek 3D menggunakan Octree
Semester II 2025/2026

Disusun oleh

Moreno Syawali Ganda Sugita	13524096
Nathaniel Christian	13524122

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

DAFTAR ISI

DAFTAR ISI..... i
BAB 1
DESKRIPSI ALGORITMA.....1
BAB II
PENGUJIAN.....7
2.1. Hasil Pengujian.....7
2.2. Hasil Pengujian Concurrency..... 10
2.3. Hasil Pengujian Viewer..... 10
LAMPIRAN..... 14

BAB 1
DESKRIPSI ALGORITMA

Algoritma *Divide and Conquer* diimplementasikan dalam *Voxelization* menggunakan struktur *Octree* dengan cara memotong kubus menjadi 8 kubus yang lebih kecil. Selanjutnya, masing-masing kubus dipotong menjadi 8 kubus yang lebih kecil lagi sampai *depth* tertentu. Setelah itu, hasil seluruh kubus digabung menjadi satu file *.obj* hasil konversi. Berikut merupakan langkah-langkah program konversi *Voxelization* dengan struktur *Octree*.

1. Divide

octree.go

```
func divideCube(cube Cube) [8]Cube {
    var child [8]Cube

    mid := cube.Size / 2
    x := cube.Min.X
    y := cube.Min.Y
    z := cube.Min.Z

    points := [8][3]float64{
        {0, 0, 0},
        {mid, 0, 0},
        {0, mid, 0},
        {mid, mid, 0},
        {0, 0, mid},
        {mid, 0, mid},
        {0, mid, mid},
        {mid, mid, mid},
    }

    for i := 0; i < 8; i++ {
        child[i] = Cube{Vertex{x+points[i][0], y+points[i][1],
z+points[i][2],}, mid}
    }

    return child
}
```

Pada fungsi *divideCube*, kubus dibagi menjadi 8 kubus yang lebih kecil dengan ukuran sisi setengah dari kubus utama. Kedelapan kubus tersebut merepresentasikan setiap sudut dari ruang kubus utama.

2. Conquer

octree.go

```
func intersectCubeTriangle(c Cube, triangles []Triangle) bool {
    cubeBox := cubeToBox(c)

    for _, t := range triangles {
        triBox := getBoundingTriangle(t)
```

```
        if isIntersect(cubeBox, triBox) {
            return true
        }
    }

    return false
}

func buildTree(cube Cube, triangles []Triangle, currDepth int, maxDepth
int, done map[int]int, remove map[int]int) *OctreeNode {
    if !intersectCubeTriangle(cube, triangles) {
        if currDepth > 0 && remove != nil {
            remove[currDepth]++
        }
        return nil
    }

    node := &OctreeNode{Cube: cube, triangles: triangles}
    if currDepth > 0 && done != nil {
        done[currDepth]++
    }

    // base case
    if currDepth == maxDepth {
        node.isLastNode = true
        return node
    }

    // divide
    dividedCubes := divideCube(cube)

    for i := 0; i < 8; i++ {
        node.Child[i] = buildTree(dividedCubes[i], triangles,
currDepth+1, maxDepth, done, remove)
    }

    return node
}
```

Fungsi *buildTree*, mengevaluasi kedelapan sub-kubus satu per satu dengan langkah sebagai berikut.

- Pruning
  Fungsi memanggil *intersectCubeTriangle* untuk mengecek apakah sub-kubus berpotongan dengan segitiga yang ada di model 3D.
- Base Case
  Jika sub-kubus tidak bersinggungan dengan model 3D, maka kubus tersebut dihapus (tidak dicatat). Jika sub-kubus bersinggungan dan tingkat kedalaman sudah mencapai batas, maka sub-kubus ditetapkan sebagai *leaf node* dan dicatat untuk dimasukkan ke file *.obj*.
- Recursion

Jika sub-kubus bersinggungan dan belum mencapai batas kedalaman, fungsi memanggil *buildTree* secara rekursif untuk setiap sub-kubus untuk membaginya menjadi 8 sub-kubus yang lebih kecil.

3. Combine

octree.go

```
func getVoxel(node *OctreeNode, voxels *[]Voxel) {
    if node == nil {
        return
    }

    if node.isLastNode {
        *voxels = append(*voxels, Voxel{node.Cube})
        return
    }

    for i := 0; i < 8; i++ {
        getVoxel(node.Child[i], voxels)
    }
}
```

Setiap *parent node* yang berhasil dievaluasi akan menyimpan pointer 8 sub-kubus di dalam array *Child[8]*OctreeNode*. Setelah seluruh struktur *Octree* selesai dibangun, fungsi *getVoxel* menelusuri *Octree* kembali. Fungsi ini akan mencatat seluruh node terakhir dan menggabungkannya ke dalam *slice voxels* untuk digabung menjadi kumpulan kubus kecil yang menyusun model 3D hasil konversi.

Kami juga mengimplementasikan kedua bonus yang diberikan pada spesifikasi, yaitu *concurrency* dan *viewer* dengan penjelasan sebagai berikut.

1. *Concurrency*

conc.go

```
const MaxConcurrentDepth = 4

func buildTreeConcurrent(cube Cube, triangles []Triangle, currDepth int,
maxDepth int, done map[int]int, remove map[int]int, mu *sync.Mutex)
*OctreeNode {
    if !intersectCubeTriangle(cube, triangles) {
        if currDepth > 0 && remove != nil {
            mu.Lock()
            remove[currDepth]++
            mu.Unlock()
        }
        return nil
    }

    node := &OctreeNode{Cube: cube, triangles: triangles}
    if currDepth > 0 && done != nil {
        mu.Lock()
```

```
        done[currDepth]++
        mu.Unlock()
    }

    if currDepth == maxDepth {
        node.isLastNode = true
        return node
    }

    dividedCubes := divideCube(cube)

    if currDepth < MaxConcurrentDepth {
        var wg sync.WaitGroup
        for i := 0; i < 8; i++ {
            wg.Add(1)
            go func(index int) {
                defer wg.Done()
                node.Child[index] =
buildTreeConcurrent(dividedCubes[index], triangles, currDepth+1,
maxDepth, done, remove, mu)
            }(i)
        }
        wg.Wait()
    } else {
        for i := 0; i < 8; i++ {
            node.Child[i] = buildTreeConcurrent(dividedCubes[i],
triangles, currDepth+1, maxDepth, done, remove, mu)
        }
    }

    return node
}
```

Implementasi bonus *concurrency* bertujuan mempercepat proses konversi dengan mekanisme *Goroutine* dalam bahasa pemrograman Go. Saat kubus dibagi menjadi 8 sub-kubus, program mendistribusikan penelusuran kedelapan sub-kubus tersebut secara paralel, lalu disinkronisasi menggunakan *sync.WaitGroup* hingga seluruh cabang selesai diproses. Untuk mencegah *race condition* saat mencatat statistik, program menahan akses memori menggunakan *sync.Mutex*. Selain itu, diterapkan batasan *depth* sebesar 4 agar tidak menyebabkan *overhead* CPU dan risiko *Out of Memory*.

2. *Viewer*

viewer.go

```
func showViewer(vertices []Vertex, faces [][3]int) {
    rl.SetTraceLogLevel(rl.LogNone)
    sw, sh := 800, 600
    rl.InitWindow(int32(sw), int32(sh), "3D Object Viewer")
    defer rl.CloseWindow()
    rl.SetTargetFPS(60)

    // Calculate center point & object size
```

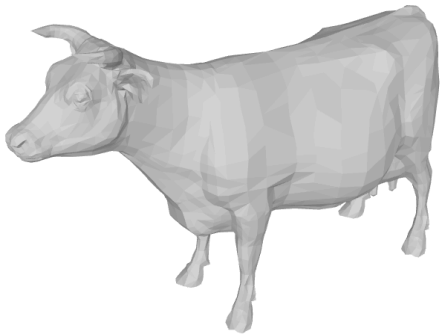
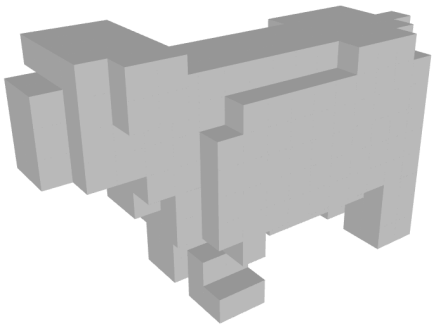
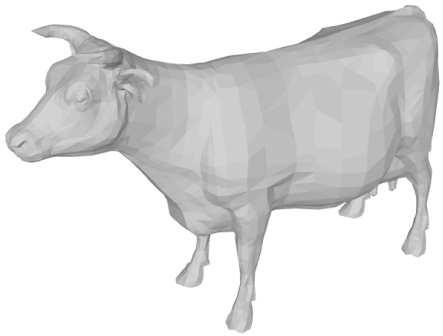
```
    var minX, minY, minZ = vertices[0].X, vertices[0].Y, vertices[0].Z
    var maxX, maxY, maxZ = minX, minY, minZ
    for _, v := range vertices {
        minX = math.Min(minX, v.X); maxX = math.Max(maxX, v.X)
        minY = math.Min(minY, v.Y); maxY = math.Max(maxY, v.Y)
        minZ = math.Min(minZ, v.Z); maxZ = math.Max(maxZ, v.Z)
    }
    cX, cY, cZ := (minX+maxX)/2, (minY+maxY)/2, (minZ+maxZ)/2

    // Calculate bounding box
    spanX, spanY, spanZ := maxX-minX, maxY-minY, maxZ-minZ
    maxSpan := math.Max(spanX, math.Max(spanY, spanZ))

    radius := maxSpan * 1.5
    if radius == 0 { radius = 10.0 }

    var yaw, pitch float64 = 0.5, 0.3

    for !rl.WindowShouldClose() {
        wheel := float64(rl.GetMouseWheelMove())
        if wheel != 0 {
            radius -= wheel * (maxSpan * 0.1)
        }

        if radius < maxSpan*0.2 { radius = maxSpan * 0.2 }

        if rl.IsMouseButtonDown(rl.MouseLeftButton) {
            delta := rl.GetMouseDelta()
            if math.Abs(float64(delta.X)) > 0.5 ||
math.Abs(float64(delta.Y)) > 0.5 {
                yaw += float64(delta.X) * 0.01
                pitch -= float64(delta.Y) * 0.01
            }
            if pitch > 1.5 { pitch = 1.5 }
            if pitch < -1.5 { pitch = -1.5 }
        }

        var toDraw []FaceToDraw
        fov := 1.5

        for _, f := range faces {
            var screenPts [3]rl.Vector2
            var avgZ float64
            visible := true

            for i := 0; i < 3; i++ {
                v := vertices[f[i]-1]
                x, y, z := v.X-cX, v.Y-cY, v.Z-cZ

                // Yaw rotation
                x1 := x*math.Cos(yaw) - z*math.Sin(yaw)
                z1 := x*math.Sin(yaw) + z*math.Cos(yaw)
                y1 := y

                // Pitch rotation
                y2 := y1*math.Cos(pitch) - z1*math.Sin(pitch)
```

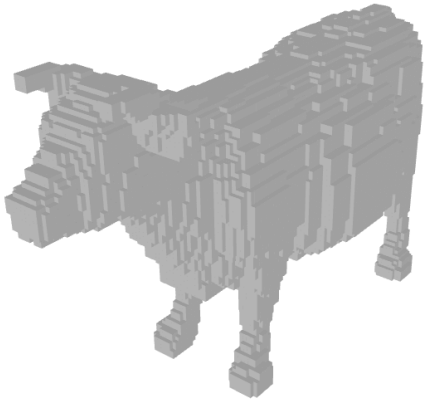
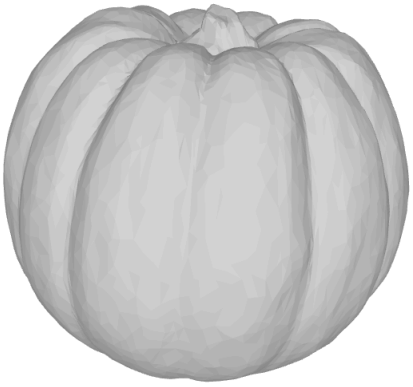
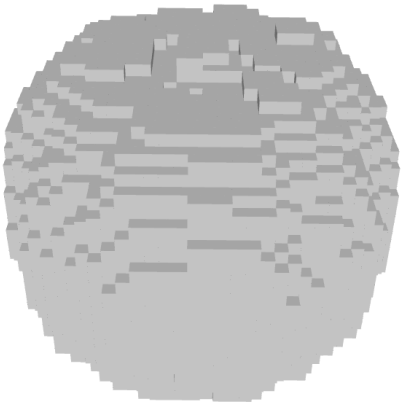
```
                z2 := y1*math.Sin(pitch) + z1*math.Cos(pitch)
                x2 := x1

                zFinal := z2 + radius

                if zFinal <= 0.1 {
                    visible = false
                    break
                }

                projX := (x2 * fov) / zFinal
                projY := (y2 * fov) / zFinal

                screenPts[i] = rl.Vector2{
                    X: float32(float64(sw)/2 + projX*float64(sh)/2),
                    Y: float32(float64(sh)/2 - projY*float64(sh)/2),
                }
                avgZ += zFinal
            }

            if visible {
                toDraw = append(toDraw, FaceToDraw{screenPts[0],
screenPts[1], screenPts[2], avgZ / 3.0})
            }
        }

        sort.Slice(toDraw, func(i, j int) bool {
            return toDraw[i].Depth > toDraw[j].Depth
        })

        rl.BeginDrawing()
        rl.ClearBackground(rl.RayWhite)

        for _, tri := range toDraw {
            rl.DrawTriangle(tri.P1, tri.P2, tri.P3, rl.SkyBlue)
            rl.DrawTriangle(tri.P1, tri.P3, tri.P2, rl.SkyBlue)
            rl.DrawLineV(tri.P1, tri.P2, rl.DarkBlue)
            rl.DrawLineV(tri.P2, tri.P3, rl.DarkBlue)
            rl.DrawLineV(tri.P3, tri.P1, rl.DarkBlue)
        }

        rl.EndDrawing()
    }
}
```

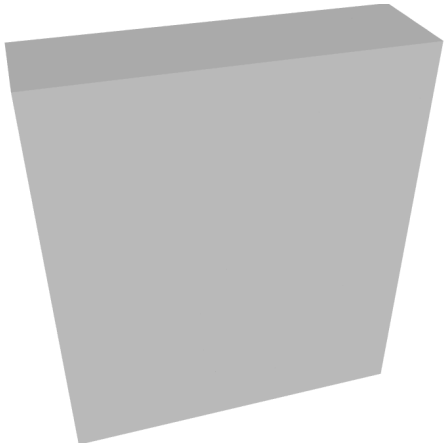
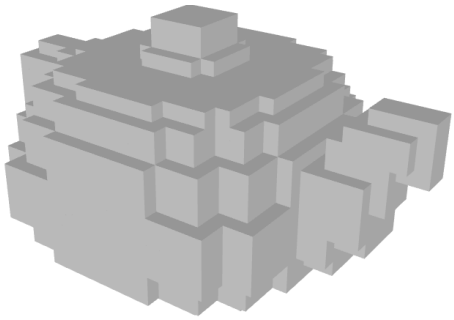
Program *viewer.go* berfungsi sebagai *rasterizer* 3D manual untuk memproyeksikan koordinat ruang 3D menjadi piksel 2D menggunakan perhitungan aljabar linear. Program menggunakan perhitungan matriks *Perspective Projection* dan *Auto-Fit Camera* berdasarkan *bounding box* objek. Sistem kamera menggunakan kalkulasi trigonometri orbital (Yaw dan Pitch) yang memungkinkan kamera berotasi pada titik tengah objek. Program juga menerapkan *Painter's Algorithm* dengan *Z-depth sorting* untuk menggambar poligon dari yang terjauh hingga terdekat.

BAB II
PENGUJIAN

2.1. Hasil Pengujian

No.	File	Depth	Visual	Statistik
1.	cow.obj	4	 	Banyaknya voxel yang terbentuk: 384 Banyaknya vertex yang terbentuk: 3072 Banyaknya faces yang terbentuk: 4608 Statistik node octree yang terbentuk: 1 : 4 2 : 20 3 : 91 4 : 384 Statistik node yang tidak perlu ditelusuri: 1 : 4 2 : 12 3 : 69 4 : 344 Kedalaman octree: 4 Lama waktu program berjalan: 156.3518ms Path output: test/cow-voxed.obj
2.	cow.obj	7		Banyaknya voxel yang terbentuk: 48645 Banyaknya vertex yang terbentuk: 389160 Banyaknya faces yang terbentuk: 583740 Statistik node octree yang terbentuk: 1 : 4 2 : 20 3 : 91 4 : 384 5 : 1754 6 : 8822 7 : 48645

				Statistik node yang tidak perlu ditelusuri: 1 : 4 2 : 12 3 : 69 4 : 344 5 : 1318 6 : 5210 7 : 21931 Kedalaman octree: 7 Lama waktu program berjalan: 15.4305684s Path output: test/cow-voxed.obj
3.	pumpkin.obj	5	 	Banyaknya voxel yang terbentuk: 6366 Banyaknya vertex yang terbentuk: 50928 Banyaknya faces yang terbentuk: 76392 Statistik node octree yang terbentuk: 1 : 8 2 : 58 3 : 274 4 : 1315 5 : 6366 Statistik node yang tidak perlu ditelusuri: 1 : 0 2 : 6 3 : 190 4 : 877 5 : 4154 Kedalaman octree: 5 Lama waktu program berjalan: 4.476354s Path output: test/pumpkin-voxed.obj

4.	line.obj	3	 	Banyaknya voxel yang terbentuk: 112 Banyaknya vertex yang terbentuk: 896 Banyaknya faces yang terbentuk: 1344 Statistik node octree yang terbentuk: 1 : 4 2 : 16 3 : 112 Statistik node yang tidak perlu ditelusuri: 1 : 4 2 : 16 3 : 16 Kedalaman octree: 3 Lama waktu program berjalan: 13.7777ms Path output: test/line-voxed.obj
5.	teapot.obj	5	 	Banyaknya voxel yang terbentuk: 2973 Banyaknya vertex yang terbentuk: 23784 Banyaknya faces yang terbentuk: 35676 Statistik node octree yang terbentuk: 1 : 4 2 : 23 3 : 110 4 : 560 5 : 2973 Statistik node yang tidak perlu ditelusuri: 1 : 4 2 : 9 3 : 74

				4 : 320 5 : 1507 Kedalaman octree: 5 Lama waktu program berjalan: 317.5923ms Path output: test/teapot-voxed.obj

Berdasarkan hasil pengujian, program berhasil mengonversikan file *.obj* dengan *Voxelization* dalam struktur *Octree* dengan hasil dan waktu yang dibutuhkan sesuai.

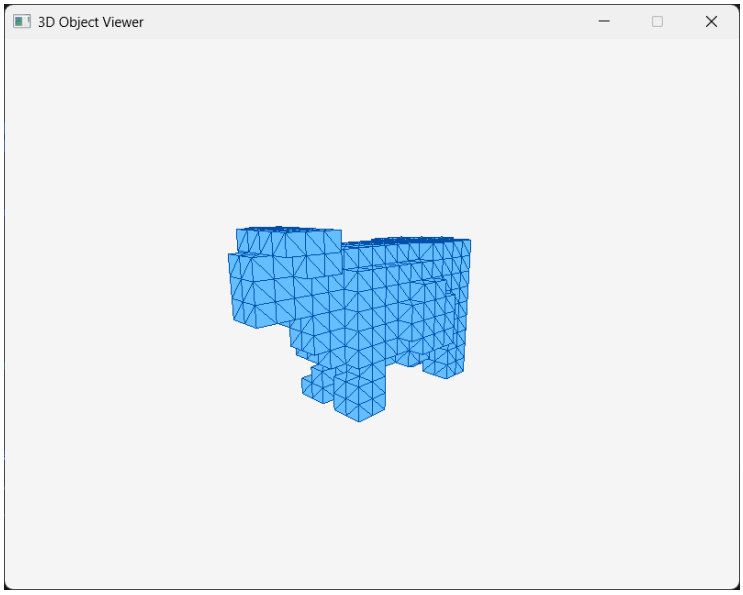
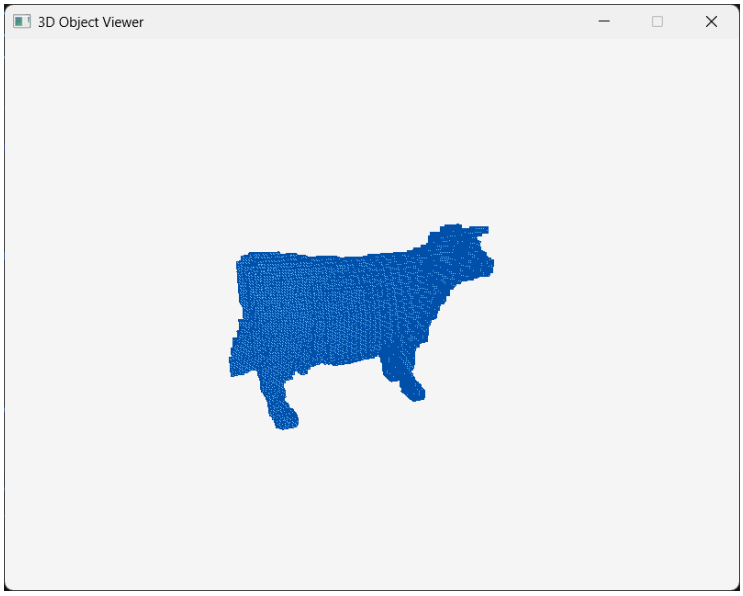
2.2. Hasil Pengujian *Concurrency*

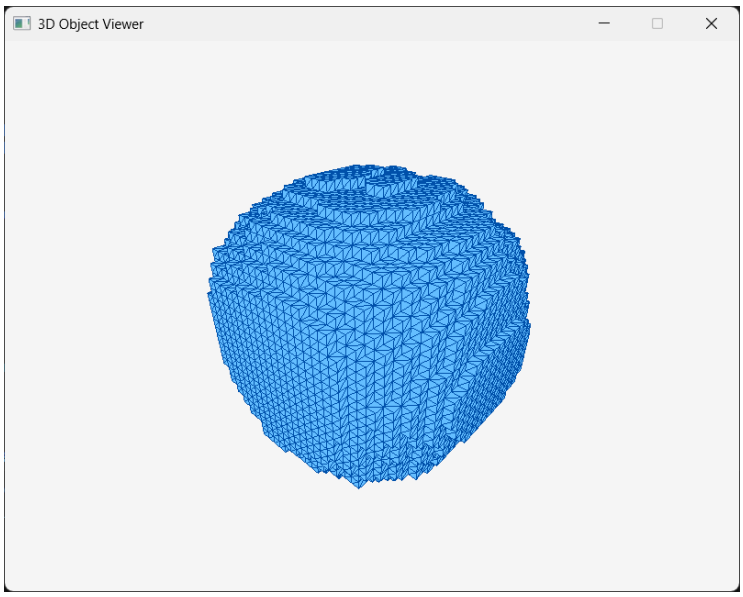
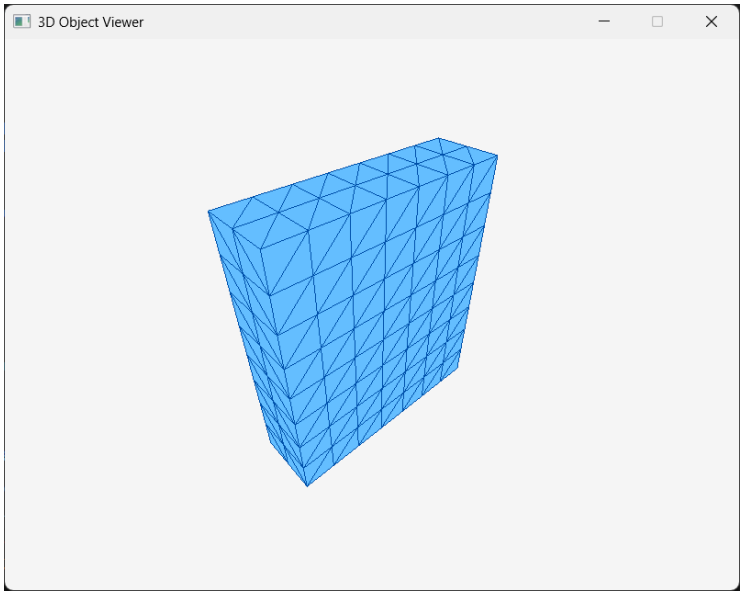
No.	File	Depth	Waktu Sekuensial	Waktu Konkuren
1.	cow.obj	4	156.3518ms	62.5314ms
2.	cow.obj	7	15.4305684s	4.4072587s
3.	pumpkin.obj	5	4.476354s	811.435ms
4.	line.obj	3	13.7777ms	23.9603ms
5.	teapot.obj	5	317.5923ms	258.0868ms

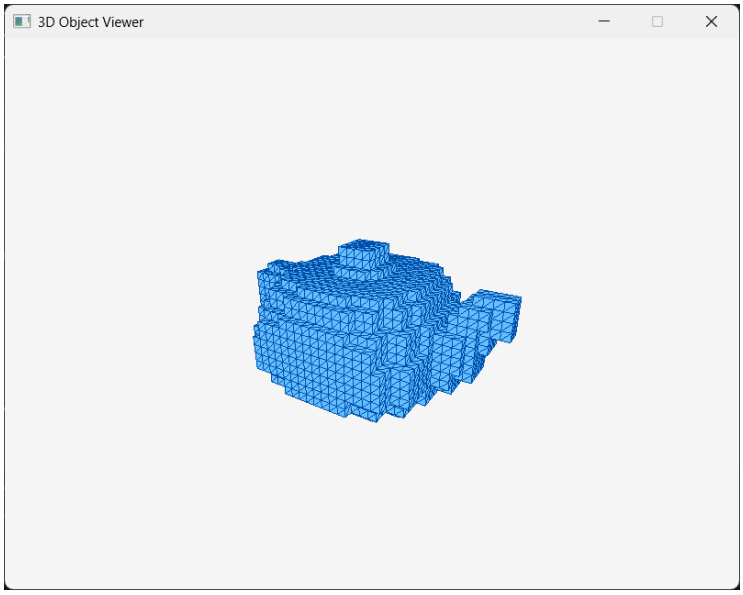
Berdasarkan hasil pengujian, *Concurrency* efektif untuk memproses pada model yang kompleks dan *depth* yang besar, sementara malah lebih lambat daripada metode sekuensial untuk model yang sederhana dengan *depth* yang kecil. Hal ini disebabkan karena adanya *overhead* untuk membuat *thread* baru.

2.3. Hasil Pengujian *Viewer*

No.	File	Depth	Hasil Viewer

1.	cow.obj	4	
2.	cow.obj	7	

3.	pumpkin.obj	5	
4.	line.obj	3	

5.	teapot.obj	5	

Program *viewer* berhasil dalam menampilkan model objek dengan baik. Namun, dengan model yang kompleks dan *depth* yang besar, *viewer* menjadi tidak responsif dan berisiko mengalami *crash*.

LAMPIRAN

Tautan Repository Github: https://github.com/Vearance/Tucil2_13524096_13524122.git

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan dapat mengembalikan output	✓	
6	[Bonus] Program menggunakan concurrency	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj	✓	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

(Moreno Syawali Ganda Sugita)

(Nathaniel Christian)